In the Name of the Creator of Beauty



Final Project Report

Introduction to Artificial Intelligence

Instructor:

Dr. Shahabeddin Nabavi

Student Information:

Maral Esmikhani

Industrial Engineering

Winter 1403 (Solar Hijri calendar)

1. Importing libraries

In the first section, all the libraries needed while writing the code were imported.

2. Loading the dataset

Then the file named `dataset` was loaded.

3. Features, target, and column types

Since the dataset consisted of features and a label, I considered the `label` as the `target` and categorized the columns into two groups: `numericals` and `categoricals`.

4. Checking for missing values

Now, following the first task of the project, we check how much missing data exists. Here we use the `isnull` method, and we can see that there is no missing (empty) data in this dataset.

5. Imputation

Nevertheless, I performed imputation on the data, because new data might be added to the initial dataset in the future, and some of those columns might have no value.

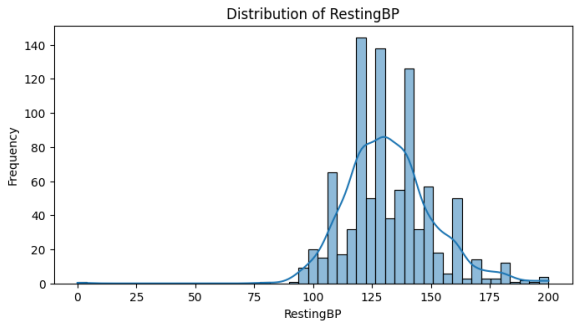
6. Distribution plots

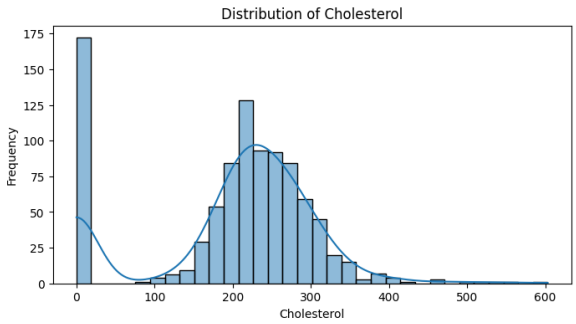
In the next step, I went on to plot the distributions of the `numerical` and `categorical` data separately, and I used their plotting methods.

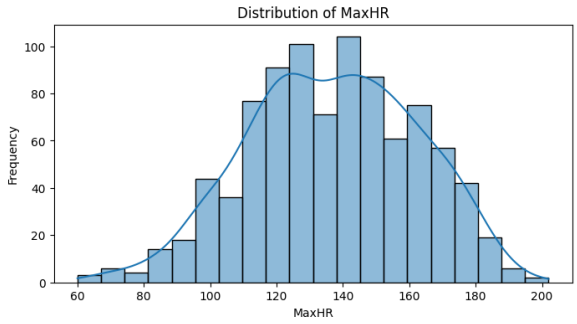
Each of these plots shows what shape the distribution of the data has for that particular category. For example, in the *Distribution of ChestPainType* plot it is shown that the `ASY` type had the largest number of samples in the dataset.

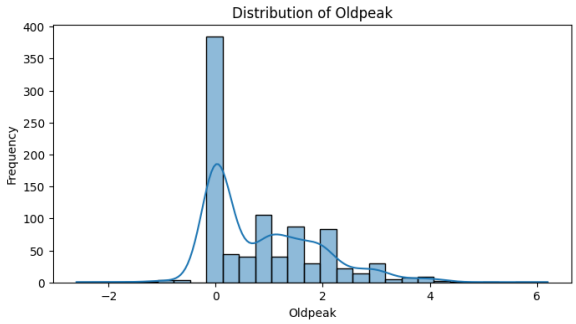
This helps us gain a better view of the data and of how they are distributed.

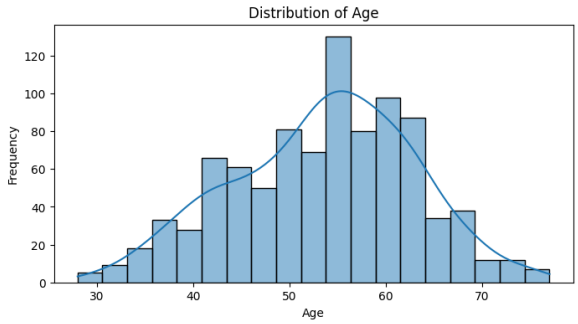
The distributions obtained are shown below:

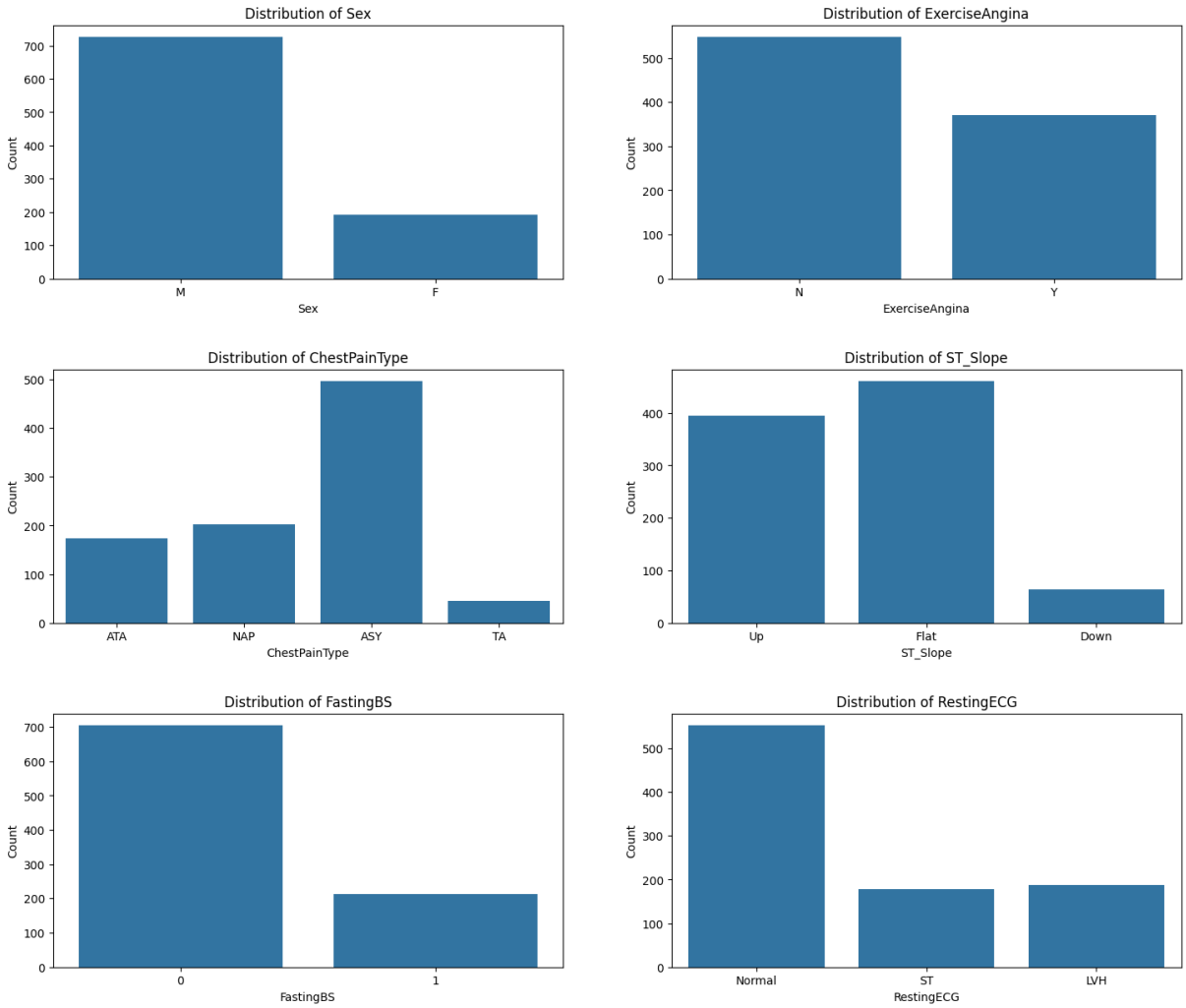

Distribution of RestingBP
Frequency
RestingBP


Distribution of Cholesterol
Frequency
Cholesterol


Distribution of MaxHR
Frequency
MaxHR


Distribution of Oldpeak
Frequency
Oldpeak


Distribution of Age
Frequency
Age



`KDE` is the abbreviation of Kernel Density Estimation, a statistical method for estimating the probability density of a continuous variable. When we use `kde=True`, we tell the program that instead of drawing an ordinary histogram, it should use the kernel density estimate (KDE) to draw the plot. This method is especially useful when we want to display the data distribution continuously and avoid discontinuous histograms.

7. Encoding string data as numerical data

In the next section we get to a task in which we must convert the data that are in `str` form into numerical data.

To do this, we need to use `LabelEncoder` for binary data and `OneHotEncoder` for non-binary string data.

Since the binary features have only two values and the rest of the data must be converted with `OneHotEncoder`, I wrote a different function for each of these two cases; the function `apply_ohe` implements the `OneHotEncoder` method.

It should be noted that the `OneHotEncoder` function was implemented first, but we had to specify in it that if the data are binary, it should ignore them, and for this we used the setting `drop="if_binary"`.

`sparse_output=False` means that we want the encoding result to be returned as an ordinary array (a dense array), i.e. the full array.

When we use `OneHotEncoder`, the output we get is of type `array`, containing the encoded data. To be able to view and process these data, and also to be able to use them again in `pandas`, we need to convert them into a `DataFrame`.

In the code I wrote for label encoding, we do not need to convert the data into a new `DataFrame`, because here the operations are performed directly on a copy of the original `DataFrame` (`df_copy`), which is itself a `pandas DataFrame`.

Therefore, we only need to define the function `apply_le` for it and use the `LabelEncoder` method.

The `target` data were numerical from the beginning, so they do not need to be changed.

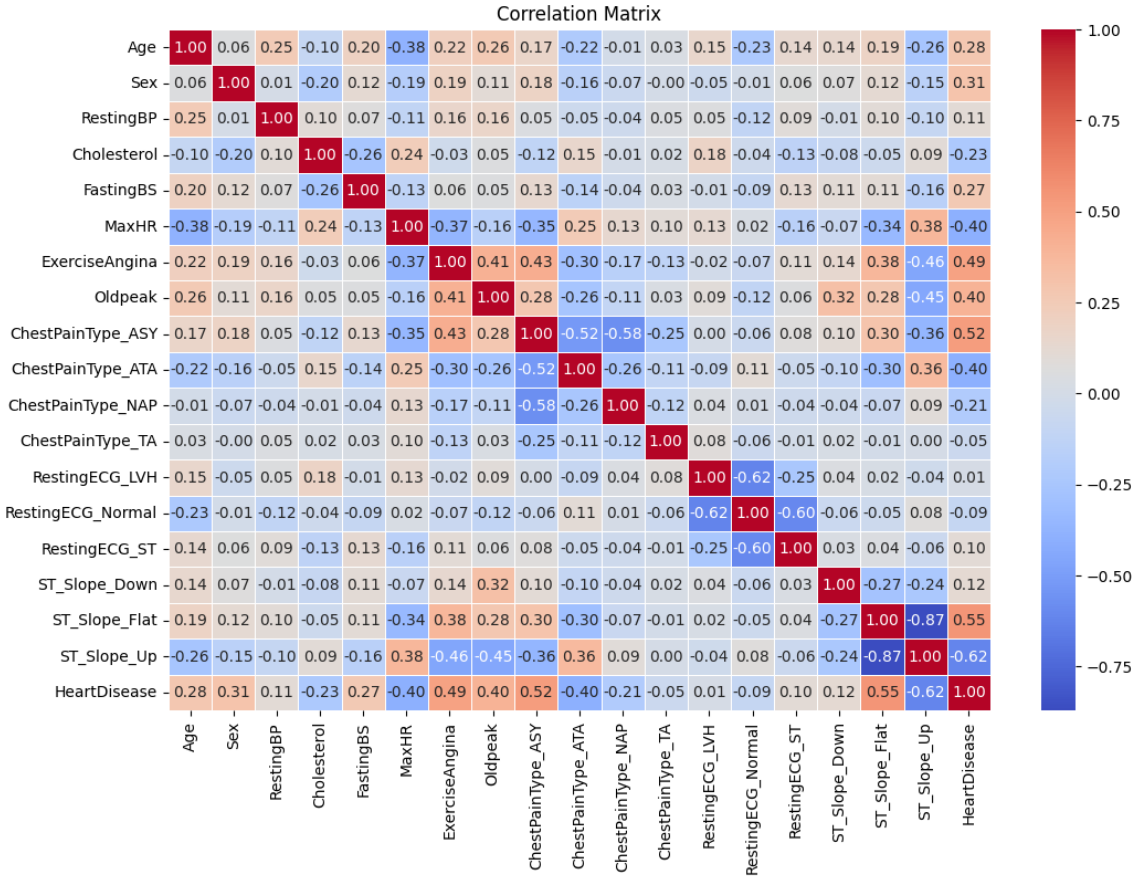
Now all of our data will be numerical.

After drawing the correlation matrix, in the next step I noticed that the `HeartDisease` column, which is our `target`, was placed in the middle of the matrix. So I moved it and shifted it to the right, so that the effect of each class on the `target` is shown more clearly.

8. Correlation matrix

By completing step 7, we will be able to examine the `correlation` between the data; it is enough to compute the correlation of the new dataset with the `correlation` method and draw it as a correlation matrix.

As is evident in the matrix, some of the features are correlated with each other, which means that the numbers obtained show that there is a relationship between these features and that they influence each other. The most important column will probably be the rightmost column, which shows the effect of each class on our `target`. What is this effect?



9. Feature scaling

In this part of the project, I decided to use standard scaling (`StandardScaler`) for the feature data, so that all features are on the same scale. This helps machine learning models perform better, because some algorithms, such as linear regression or distance-based models (like K-Nearest Neighbors), are sensitive to the scale of the data.

First, I split the data into two parts: features (`X`) and target (`y`). Here, the `HeartDisease` column, which is the target variable, was separated from the features and taken as `y`. The features in `X` include all columns except `HeartDisease`.

To scale the features, I used the `StandardScaler` class from the `sklearn.preprocessing` library. `StandardScaler` works by setting the mean of each feature to zero and its standard deviation to one. This puts the features on the same scale and makes the effect of each feature on the model more balanced.

After separating the features and the target, I performed the scaling on the features (`X`). Using the `fit_transform()` method, I converted the `X` data to the standard scale.

Finally, the scaled data, which were returned as a `NumPy` array, were converted into a `DataFrame` so that I could easily manage and examine them. This also preserved the

column names, which is useful for later analysis.

10. Splitting the data into training and test sets

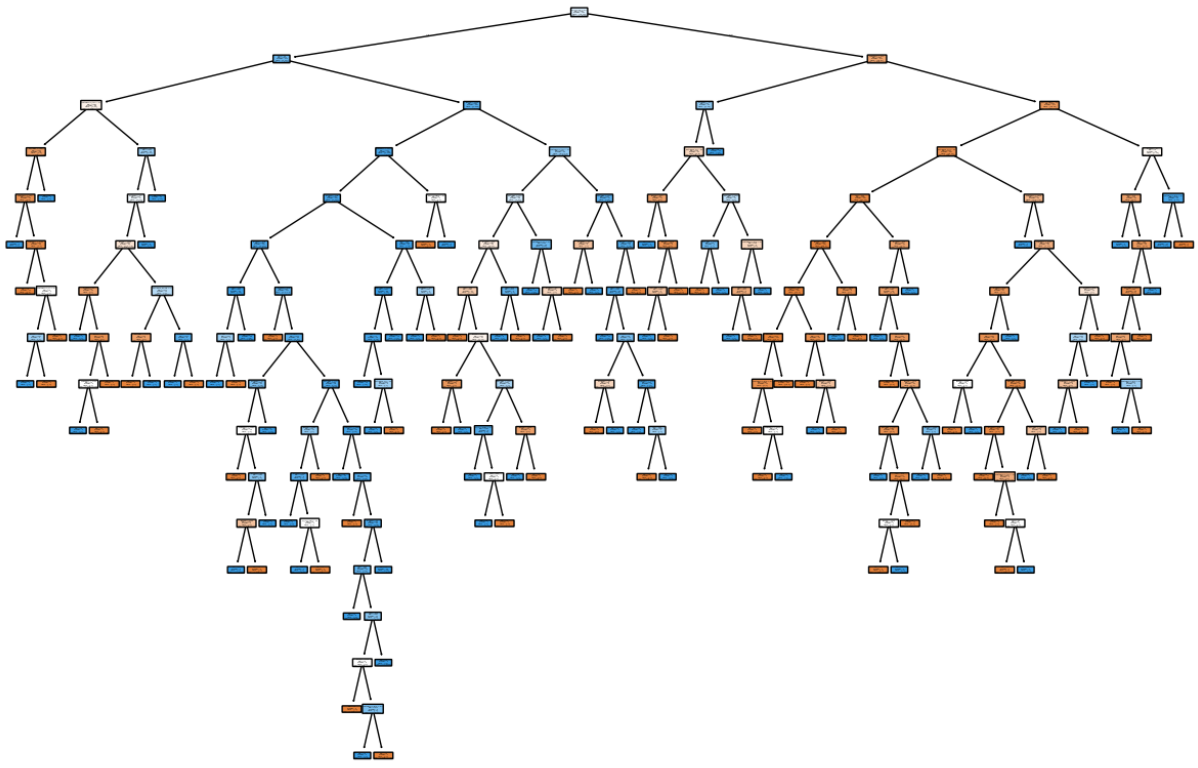
First, I split the data into training and test sets. This is done to evaluate the model's performance on new and unseen data. The split is as follows:

- The training set `train_X` and `train_y` comprises 80% of the data.
- The test set `test_X` and `test_y` comprises 20% of the data.

For this split, I used the `train_test_split` function from the `sklearn.model_selection` library. In addition, to ensure the reproducibility of the results, the parameter `random_state=42` was chosen for randomizing the data. The reason for choosing 42 is that this number is commonly used by programmers for this purpose.

Continuing this section, I used the decision tree algorithm to train the model. This algorithm is one of the supervised learning models and is particularly used for classification problems.

The decision tree model was built using `DecisionTreeClassifier` from the `sklearn.tree` library. The parameter `random_state=42` was chosen to set the randomization of the tree and to ensure the reproducibility of the results.



After building the model, I used the `fit()` method to train it on the training set (`X_train`, `y_train`).

In machine learning, the `fit()` method is used to train the model and learn its parameters from the data, whereas `fit_transform()` is a combination of `fit()` and `transform()`, which simultaneously trains the model and applies the necessary changes to the data. On the other hand, `transpose()` is used to rotate the dimensions of a matrix or array, in the sense that it converts rows into columns and columns into rows. Therefore, `fit()` and `fit_transform()` relate to training the model and processing the data, whereas `transpose()` relates to changing the structure of the data.

After training the model, I made predictions for the test set (`X_test`). The predictions were obtained using the `predict()` method.

Then the accuracy of the model was calculated and displayed using the `accuracy_score` function from the `sklearn.metrics` library.

For a more precise evaluation of the model's performance, I used the classification report. This report includes the following metrics:

- **Precision:** the percentage of correct predictions for each class.
- **Recall:** the percentage of correctly identified cases out of all actual cases.
- **F1-Score:** the harmonic mean of precision and recall, which is very useful for imbalanced models.

This report was generated using the `classification_report()` function from the `sklearn.metrics` library.

Confusion Matrix:

To evaluate the model's performance more precisely, I used the confusion matrix. This matrix shows the number of correct and incorrect predictions for each class and helps to analyze the model's performance further. This matrix was obtained using the `confusion_matrix()` function from the `sklearn.metrics` library.

Finally, to visualize how the decision tree model makes its decisions, I used the `plot_tree()` function. This function displays the decision tree graphically, showing the features and the model's decisions based on the nodes. The parameter `filled=True` was set for coloring the nodes, and `class_names` was set for naming the classes. This figure was displayed using `matplotlib`, with the figure size set to 15×10.

11. Extracting feature importance

At first, using the `feature_importances_` attribute of the decision tree model trained on the training set, I extracted the importance that the model assigned to each feature. This attribute is an array in which each value indicates the effect of that feature on the model's decision-making.

The importance of each feature is usually in the range $[0, 1]$, and the sum of the importances of all features equals 1. Features with higher importance usually play a key role in splitting the data.

Then, I put these data in a `DataFrame` containing two columns:

- **Feature:** the feature names.
- **Importance:** the importance assigned to each feature by the decision tree model.

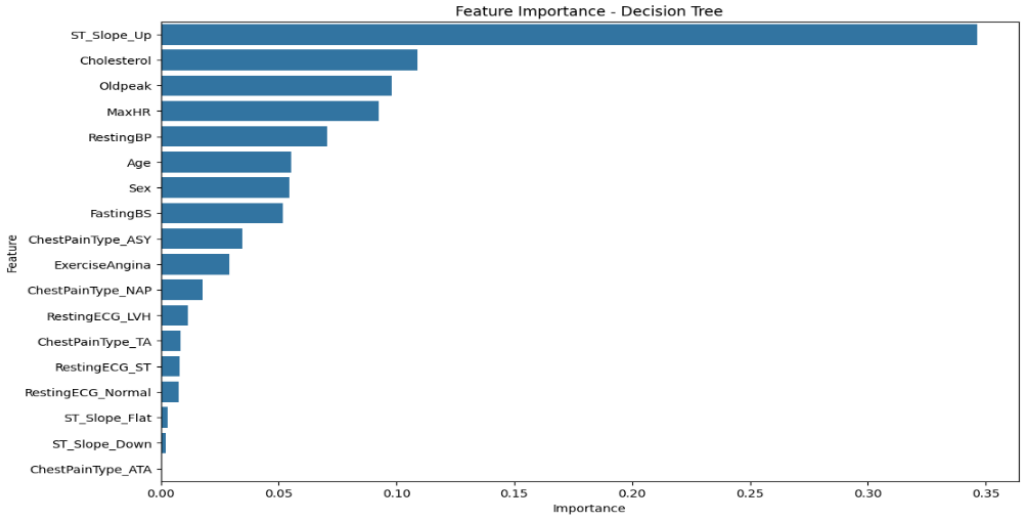
This `DataFrame` allowed me to easily process the data and display the features. To place the features with the greatest effect at the top of the table, I sorted the `DataFrame` by importance value. This allowed me to identify the features with the greatest effect.

Then, I used the top 10 features in terms of importance for the model's prediction. These features are the most important parameters that the decision tree used to split the data.

Feature importance:

To better display and compare the features, I used a bar chart. This chart displays the importance of each feature visually, and using `seaborn` I drew the chart in a suitable and readable size.

The horizontal axis (x) shows the importance of the features. The vertical axis (y) shows the names of the features.



12. Dimensionality reduction with PCA

In this section, I used Principal Component Analysis (PCA) to reduce the dimensionality of the data. PCA is a popular dimensionality reduction technique that retains the main features of the data in fewer dimensions, while preserving the maximum variance present in the data.

The parameter `n_components=2` means that the data were compressed into two principal components, `PC1` and `PC2`.

The input data, which had been scaled (`X_scaled`), were converted into two new dimensions.

After applying PCA, I put the compressed data in a `DataFrame` named `pca_df`. This `DataFrame` includes:

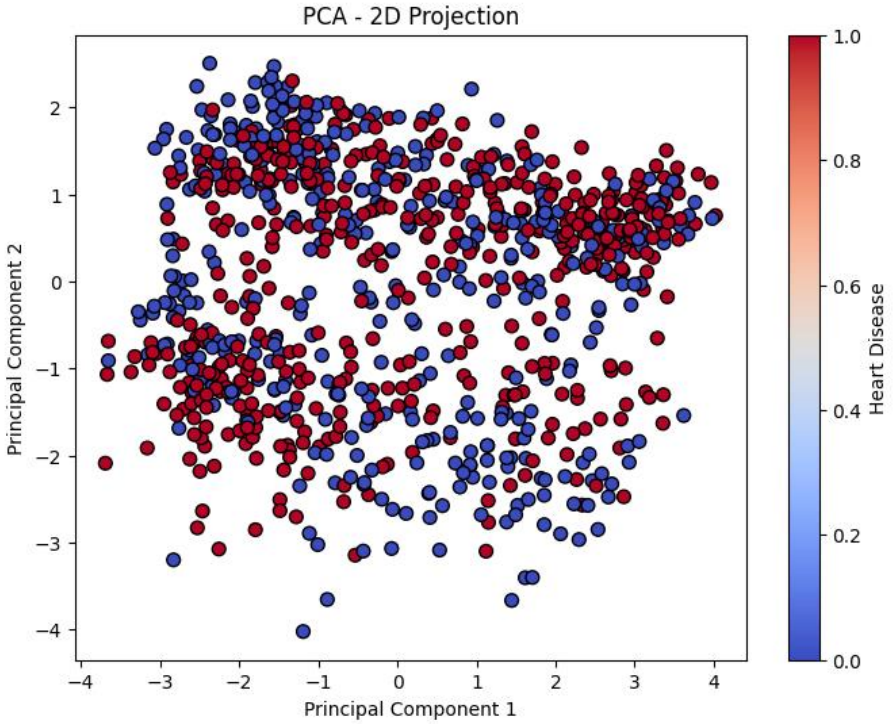
- Two columns, `PC1` and `PC2`, which represent the first and second principal components.
- Another column named `HeartDisease`, which I added; it shows the target (presence or absence of heart disease), so that the relationship between the compressed data and the target variable can be observed.

To observe the PCA results visually, I used a scatter plot. The horizontal axis represents `PC1` and the vertical axis represents `PC2`.

The points are colored according to the presence or absence of heart disease. The colors were taken from `cmap='coolwarm'`, which changes the colors from blue (for absence of disease) to red (for presence of disease).

In addition, I used a `colorbar` to display the colors and their relationship with heart disease.

Finally, we printed the explained variance ratio of each of the principal components `PC1` and `PC2`. These values show how much of the total variance of the data each component explains. This information helps us understand how much of the original information each component has preserved.



13. Clustering with K-Means

To examine the effect of the number of clusters, I used a range of cluster counts (from 2 to 5 clusters).

For each number of clusters, the `KMeans` model was run and the different clusters of the data were identified. Then the cluster labels were added to the data, and the number of samples in each cluster was displayed. This information allowed me to see how the data are divided among the different clusters.

For example:

Using `pd.Series(kmeans.labels_).value_counts()`, the number of samples in each cluster is printed. These results show how many data points each cluster contains.

Dimensionality reduction with PCA:

After performing clustering, for better analysis and visualization of the data, I used PCA (Principal Component Analysis) to reduce the dimensionality to three. This helped me to see the clusters in three-dimensional space and to analyze them.

PCA with 3 principal components was applied (`n_components=3`) so that I could display the data effectively in 3D space. This helped make the differences between the clusters and their distribution in the data space visually observable.

Displaying the results in 3D:

For each number of clusters, I used a 3D plot to visualize the PCA results. In this plot:

The X, Y, and Z axes represent the three principal components of PCA, which show the reduced dimensions of the data.

Each point in this plot represents a sample, and the colors indicate the different clusters to which the samples belong. The colors are chosen automatically by the `viridis` palette.

The cluster labels are indicated in this plot with the help of colors.

Using `fig.colorbar(scatter, label='Cluster')`, a color bar was added that shows the different clusters.

Explained variance ratio of PCA:

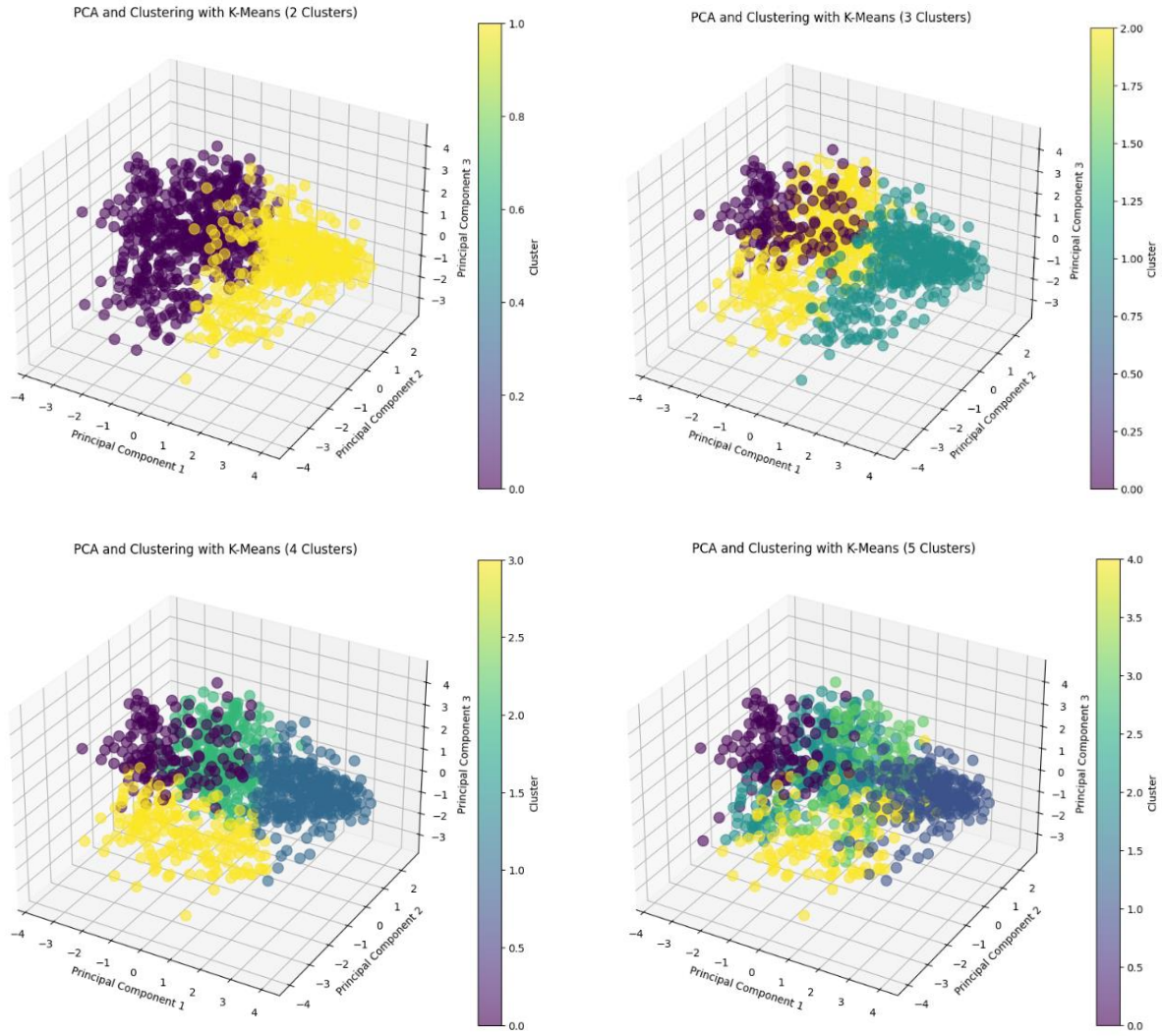
After applying PCA, I printed the explained variance ratio of each principal component for each number of clusters. This information helped me understand how much of the data's variance each component explains and how much of the original information is preserved.

These results were obtained using `pca.explained_variance_ratio_`.

Results:

For each number of clusters, the number of samples in each cluster, the explained variance ratio, and 3D plots for displaying the clusters were provided.

Analyzing the results and comparing the 3D plots helped me understand how the clustering changes as the number of clusters increases, and whether the partitioning of the data improves or not.



14. Classification models

14.1. Decision Tree

First, the decision tree model was built using the `DecisionTreeClassifier` class from the `sklearn.tree` library. To prevent the results from being random, the parameter `random_state=42` was used.

After training the model on the training data (`X_train`, `y_train`), predictions were made on the test data (`X_test`) and the model's accuracy was calculated.

In addition to accuracy, the classification report of the decision tree model was printed, which includes metrics such as precision, recall, and F1-Score.

Regression:

The logistic regression model was created and trained using the

`LogisticRegression` class from `sklearn.linear_model`. Similar to the decision tree, `random_state=42` was used for the stability of the results.

The model's predictions on the test data were made and the model's accuracy was calculated.

The classification report of this model was also displayed, so that a better evaluation of its performance is available.

XGBoost:

The `XGBoost` model (an advanced algorithm for classification and regression) from the `xgboost` library was used.

Similar to the previous models, this model was also trained on the training data and its predictions were made on the test data.

The model's accuracy was calculated and its classification report was printed.

SVM:

The `SVM` model from `sklearn.svm` was used for classification. This model was also created and trained with `random_state=42`.

The predictions and the model's accuracy were calculated, and the classification report of the SVM was displayed.

Finally, the accuracies of the models were printed, which shows the percentage of correct predictions of each model relative to all the test data.

Also, for each model, a classification report was provided, which includes metrics such as:

- **Precision:** the accuracy of the positive predictions.
- **Recall:** the model's ability to identify all positive cases.
- **F1-Score:** the harmonic mean of precision and recall, which is useful for problems with imbalanced classes.

This section compares the accuracy and overall performance of the different models. This comparison can help in choosing the best model for the available data.

The models were also compared in terms of accuracy and classification report, in order to determine which model performs best in predicting heart disease.

15. Model evaluation

First, I put the models in a dictionary so that I could easily use them. For each model, training on the training data (`train_X`, `train_y`) was performed first, and then the models made predictions on the test data (`test_X`). To evaluate the performance precisely, several different metrics were calculated for each model.

Evaluation metrics:

To evaluate the performance of each model, I used the following metrics:

- **Accuracy:** the percentage of correct predictions relative to all the test data.
- **Precision:** the model's ability to correctly identify positive samples.
- **Recall:** the model's ability to identify all positive samples.
- **F1-Score:** the harmonic mean of precision and recall, which is very useful for models with imbalanced data.
- **Area Under the Curve (AUC):** the area under the ROC curve of the model in separating the classes, which shows the model's ability to distinguish between the positive and negative classes.
- **Execution Time:** the time spent on training the model and making the predictions.

Model evaluation:

For each model, first the execution time of training and prediction was recorded. Then, the predictions were made, and the accuracy, precision, recall, F1-Score, and AUC were calculated.

For the AUC evaluation, the ROC curve was drawn and the area under the curve was calculated. This metric shows how well the model can separate the different classes (for example, presence or absence of heart disease).

For each model, the classification report was printed, which includes the precision, recall, and F1-Score for each class.

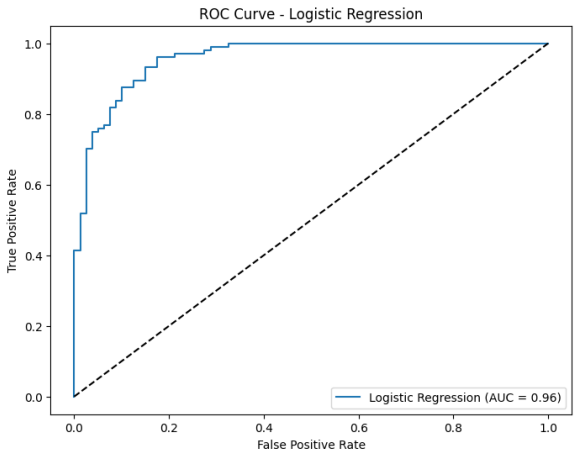
ROC plots:

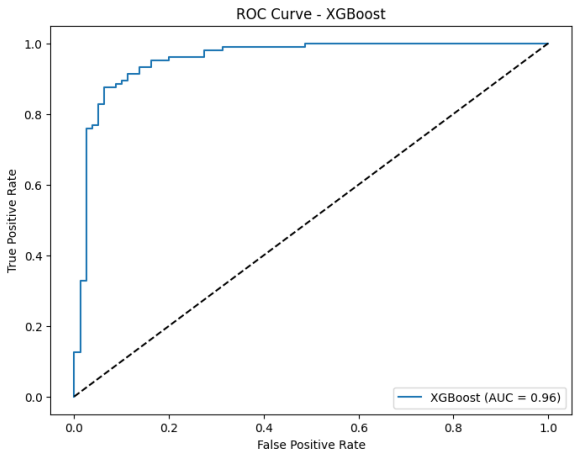
For the visual evaluation of each model, the ROC Curve was plotted. In this plot:

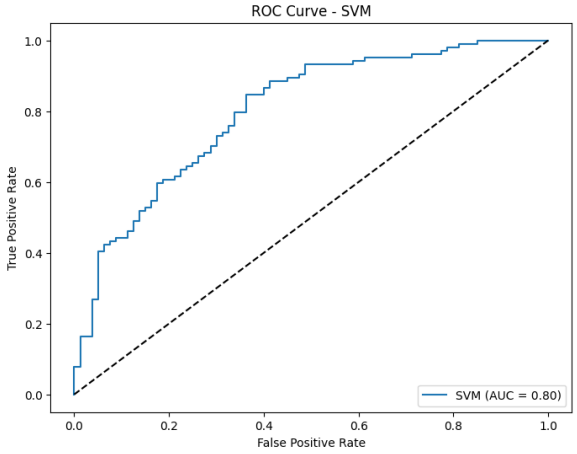
The horizontal axis is the False Positive Rate (FPR) and the vertical axis is the True Positive Rate (TPR).

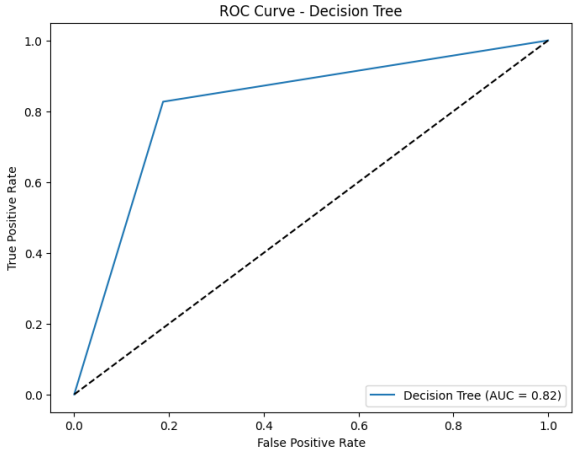
The diagonal line in this plot represents random predictions.

Using AUC, the models can be compared in terms of their ability to separate the classes.









Model comparison:

After evaluating each model, the results were stored in a `DataFrame`, and the models were sorted by AUC. This allowed me to identify the best model in terms of its ability to separate the classes.

Finally, the best model based on AUC was identified and printed. This model was found to be Logistic Regression.